Project Title

Automated Project Task Planning and Scheduling

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Md Aman - 2520030190
V. Sree Pragnya - 2520030238
Vaishnavi - 2520030247

Team Number:

Section: 04

Under the guidance of

Yamini Devi

Assistant Professor, CSE



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "**Automated Project Task Planning and Scheduling**" is a record of Bonafide work of **MD AMAN – 2520030190, V SREE PRAGNYA –2520030238, Vaishnavi - 2520030247** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Md Aman - 2520030190
V. Sree Pragnya - 2520030238
Vaishnavi - 2520030247

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "**Automated Project Task Planning and Scheduling**" is a bonafide work done and submitted by **MD AMAN – 2520030190, V SREE PRAGNYA – 2520030238, Vaishnavi - 2520030247** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Yamini Devi**, **Assistant Professor**, **CSE** which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Md Aman - 2520030190
V. Sree Pragnya - 2520030238
Vaishnavi - 2520030247

TABLE OF CONTENTS:

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8 – 9
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12 – 13
7.	Results & Discussion	14 – 16
8.	Conclusion & Future Scope	17
9.	References	18

1.ABSTRACT

The Automated Project Task Planning and Scheduling system is designed to organize and schedule project activities while satisfying dependencies and precedence constraints. In many projects, tasks must be completed in a specific order, and improper task planning can lead to delays, scheduling conflicts, and inefficient execution. Managing a large number of tasks manually becomes increasingly difficult as the number of dependencies grows.

The proposed system represents project tasks as vertices and their dependencies as directed edges in a graph. An adjacency list is used to store the graph efficiently, while indegree calculation helps identify tasks that can be executed first. Kahn's Topological Sorting algorithm is applied to determine a valid execution order in which every task is performed only after its required predecessor tasks are completed. The system also performs cycle detection to identify circular dependencies that prevent a valid schedule from being generated.

After determining the task order, the system uses task durations to calculate start and finish times. It further calculates task slack and identifies critical tasks and the Critical Path, which determines the minimum project completion time under the given task durations and dependency constraints.

The application provides a menu-driven terminal interface through which users can create projects, add tasks, define dependencies, view schedules, and examine project summaries. This project demonstrates the practical application of graph-based data structures and algorithms in project management, providing an organized, understandable, and efficient approach to task scheduling.

Team Name:

Team Members:

1. Md Aman - 2520030190
2. V. Sree Pragnya - 2520030238
3. Vaishnavi - 2520030247

2. INTRODUCTION

Project scheduling is an important component of project management because every project consists of multiple activities that must be planned, organized, and completed within a specific period. These activities may have different durations and dependencies, meaning that certain tasks cannot begin until other tasks have been completed. Preparing an appropriate execution order is essential for avoiding scheduling conflicts and ensuring the smooth progression of a project. As the number of tasks increases, managing these relationships manually becomes more complex and time-consuming.

Traditional manual scheduling methods may make it difficult to identify task dependencies, determine the correct execution order, estimate project completion time, and identify activities that could delay the entire project. Circular dependencies create an additional challenge because they can make it impossible to generate a valid task sequence. Therefore, an automated system is useful for representing task relationships, analysing dependencies, and preparing a structured project schedule.

The Automated Project Task Planning and Scheduling system addresses these challenges by applying Data Structures and Algorithms to project planning. Each task is represented as a vertex, and each dependency is represented as a directed edge. For example, if System Design can begin only after Requirement Analysis is completed, a directed edge is created from the Requirement Analysis task to the System Design task. The system uses an adjacency list to represent these relationships and calculates the indegree of each task to determine which tasks have no unfinished predecessors.

The application uses Kahn's Topological Sorting algorithm to generate a valid execution order. During this process, the system checks for circular dependencies by comparing the number of processed tasks with the total number of tasks. If some tasks remain unprocessed, the dependency graph contains a cycle, and the system reports that a valid schedule cannot be generated. Otherwise, the resulting topological order is used to calculate task start and finish times based on the specified durations and dependencies.

Another important feature of the system is Critical Path Analysis. The system calculates the earliest and latest start and finish times of tasks and determines their slack values. Tasks with zero slack are identified as critical tasks because delaying them can affect the project's completion time. The Critical Path represents a sequence of dependent tasks that determines the minimum project duration under the scheduling assumptions used by the system.

The application is implemented as a menu-driven Java terminal program, allowing users to create projects, enter task details, define dependencies, generate schedules, view critical tasks, and display project summaries. It uses data structures such as HashMap, ArrayList, and Queue to manage task information and execute graph algorithms.

The main objective of this project is to demonstrate how graph representation, cycle detection, Topological Sorting, and Critical Path Analysis can be applied to solve practical scheduling problems. By automating these operations, the system reduces manual scheduling effort and provides users with a clear and organized view of project execution.

3. LITERATURE SURVEY

1. Microsoft Project:

Microsoft Project is a widely used project scheduling system that manages tasks, task dependencies, durations, and project timelines. It calculates the critical path to identify tasks that directly affect the project's finish date. The system also uses task relationships to determine the scheduling order and available slack. These features are closely related to the proposed Automated Project Task Planning and Scheduling system.

Reference:

https://support.microsoft.com/en-us/project/manage-your-project-s-critical-path

2. ProjectLibre:

ProjectLibre is an open-source project management application that provides task management, dependencies, Gantt charts, and Critical Path Method scheduling. Tasks can be linked through dependencies, and changes in predecessor tasks can affect successor task dates. Its network-diagram and dependency features are similar to the graph-based scheduling approach used in this project.

Reference:

https://projectlibre.com/

3. Smartsheet Project Scheduling

Smartsheet provides project planning features for managing tasks and dependencies and visualizing schedules using Gantt charts. Its dependency functionality helps determine the sequence in which tasks should be performed. It also identifies and highlights the critical path based on task durations and predecessor relationships, making it relevant to this project.

Reference:

https://www.smartsheet.com/content/project-dependencies

4. Jira Advanced Roadmaps

Jira Advanced Roadmaps provides dependency-based scheduling for project work items. Dependencies represent relationships where one work item must be completed before another can proceed. The system displays these relationships on a timeline and can identify scheduling conflicts caused by dependent tasks. This demonstrates how dependency management can be integrated into project planning systems.

Reference:

https://support.atlassian.com/jira-software-cloud/docs/what-are-dependencies-in-advanced-roadmaps/

5. ProjectLibre Critical Path Scheduling

ProjectLibre applies the **Critical Path Method (CPM)** to calculate schedules from task dependencies. It supports linking tasks using predecessor relationships and provides network diagrams for representing task logic. The critical path is then identified from the dependency structure, which is closely related to the graph and critical-path approach used in this project.

Reference:

https://projectlibre.com/academy/linking-tasks-and-dependencies/

6. Jira Timeline Dependency Management

Jira's timeline provides a visual representation of dependencies between work items. A dependency indicates the order in which tasks need to be completed, and the timeline displays relationships between tasks. This approach is useful for understanding how task relationships affect the scheduling process and provides a reference for the dependency visualization in our system.

Reference:

https://support.atlassian.com/jira-software-cloud/docs/create-or-remove-dependencies-on-your-timeline/

7. Smartsheet Gantt and Critical Path

Smartsheet combines task dependencies with Gantt-based scheduling to automatically adjust project dates when predecessor relationships change. Its critical-path feature highlights tasks that directly affect project completion. This provides a practical example of integrating dependency management, scheduling, and critical-path analysis in one system.

Reference:

https://help.smartsheet.com/learning-track/level-3-solutions/gantt-chart-dependencies

4. Problem Statement and Objectives

Problem Statement:

Develop an automated system to generate a valid project schedule by managing task dependencies and identifying the critical path.

Problem Description:

Managing multiple tasks with dependencies manually can be difficult and may lead to incorrect task ordering or delays. The proposed system uses graph-based algorithms to organize tasks, detect circular dependencies, and generate an efficient execution schedule.

Objectives:

1. **Automate Task Scheduling:** Generate a valid execution order based on task dependencies.
2. **Manage Task Dependencies:** Represent relationships between tasks using a directed graph.
3. **Detect Circular Dependencies:** Identify cycles that can prevent a valid project schedule.
4. **Apply Topological Sorting:** Determine the correct order in which dependent tasks should be executed.
5. **Calculate Project Duration:** Calculate task start and finish times to determine the overall project completion time.
6. **Identify the Critical Path:** Find critical tasks that directly affect the project completion time.
7. **Provide an Organized Schedule:** Present the final task order and scheduling information in a simple and understandable format.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

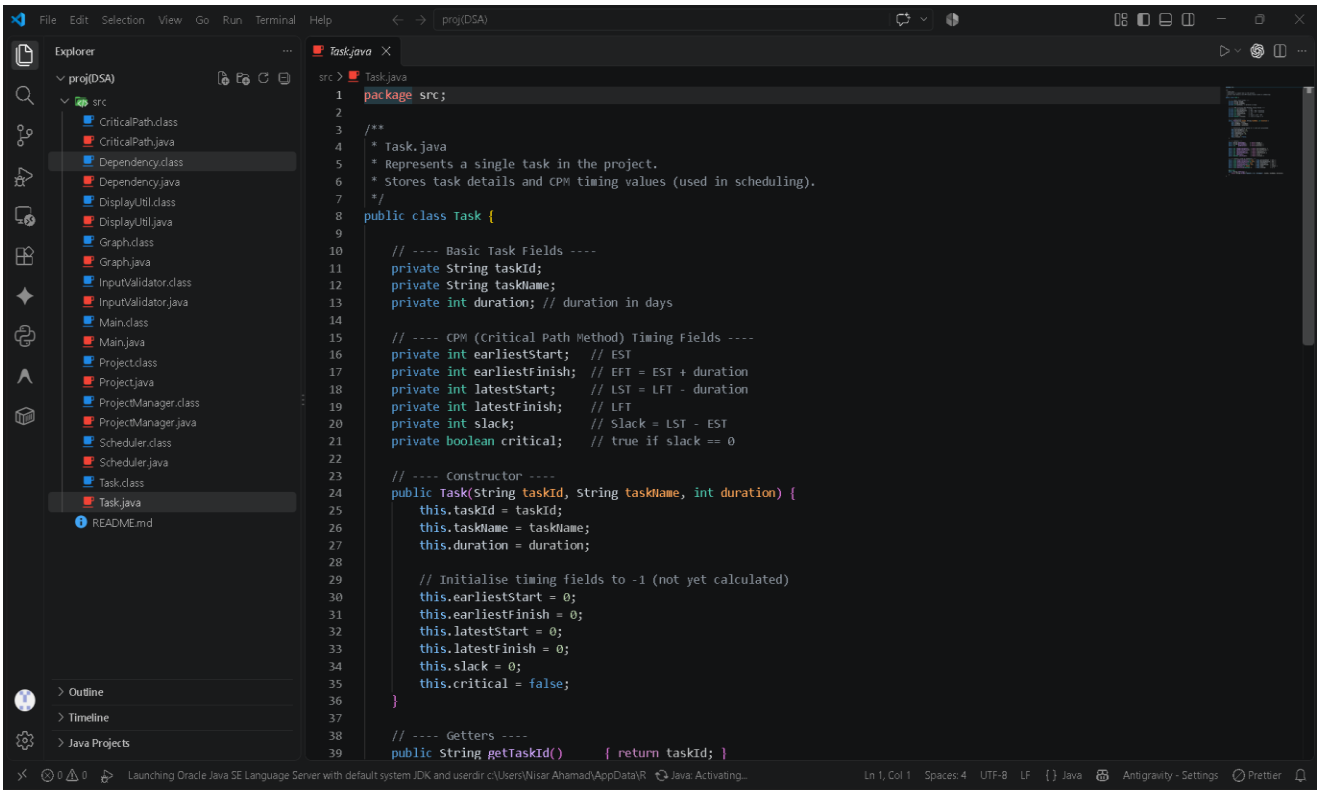
- **Processor:** Intel Core i3 or equivalent
- **RAM:** Minimum 4 GB
- **Storage:** Minimum 1 GB free space

Software Requirements:

- **Operating System:** Windows 10/11 or Linux
- **Programming Language:** Java
- **JDK:** JDK 17 or later
- **IDE/Editor:** VS Code, IntelliJ IDEA, or Eclipse

6. IMPLEMENTATION

Give here 3 to 4 modules code of your project



```java
package src;

/**
 * Task.java
 * Represents a single task in the project.
 * Stores task details and CPM timing values (used in scheduling).
 */
public class Task {

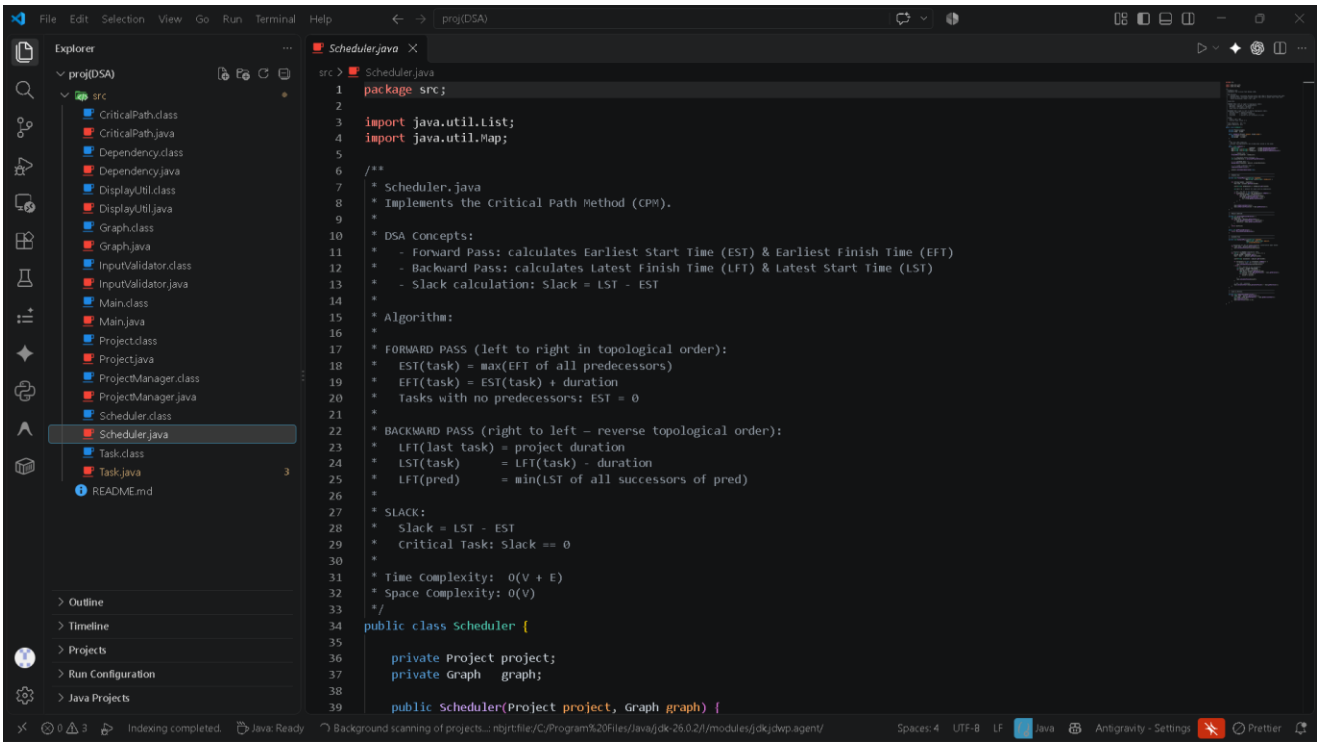
    // ---- Basic Task Fields ----
    private String taskId;
    private String taskName;
    private int duration; // duration in days

    // ---- CPM (Critical Path Method) Timing Fields ----
    private int earliestStart;   // EST
    private int earliestFinish;  // EFT = EST + duration
    private int latestStart;     // LST = LFT - duration
    private int latestFinish;    // LFT
    private int slack;           // Slack = LST - EST
    private boolean critical;    // true if slack == 0

    // ---- Constructor ----
    public Task(String taskId, String taskName, int duration) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.duration = duration;

        // Initialise timing fields to -1 (not yet calculated)
        this.earliestStart = 0;
        this.earliestFinish = 0;
        this.latestStart = 0;
        this.latestFinish = 0;
        this.slack = 0;
        this.critical = false;
    }

    // ---- Getters ----
    public String getTaskId()      { return taskId; }
```



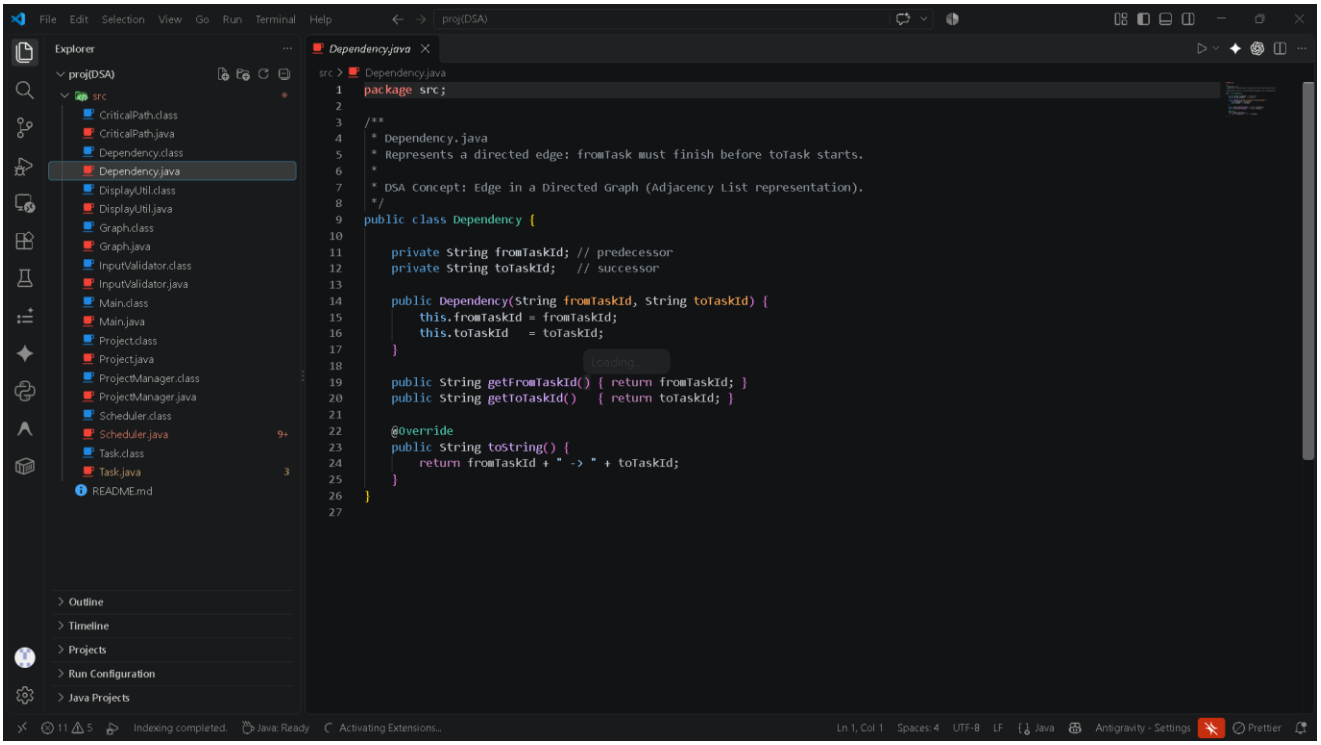
```java
package src;

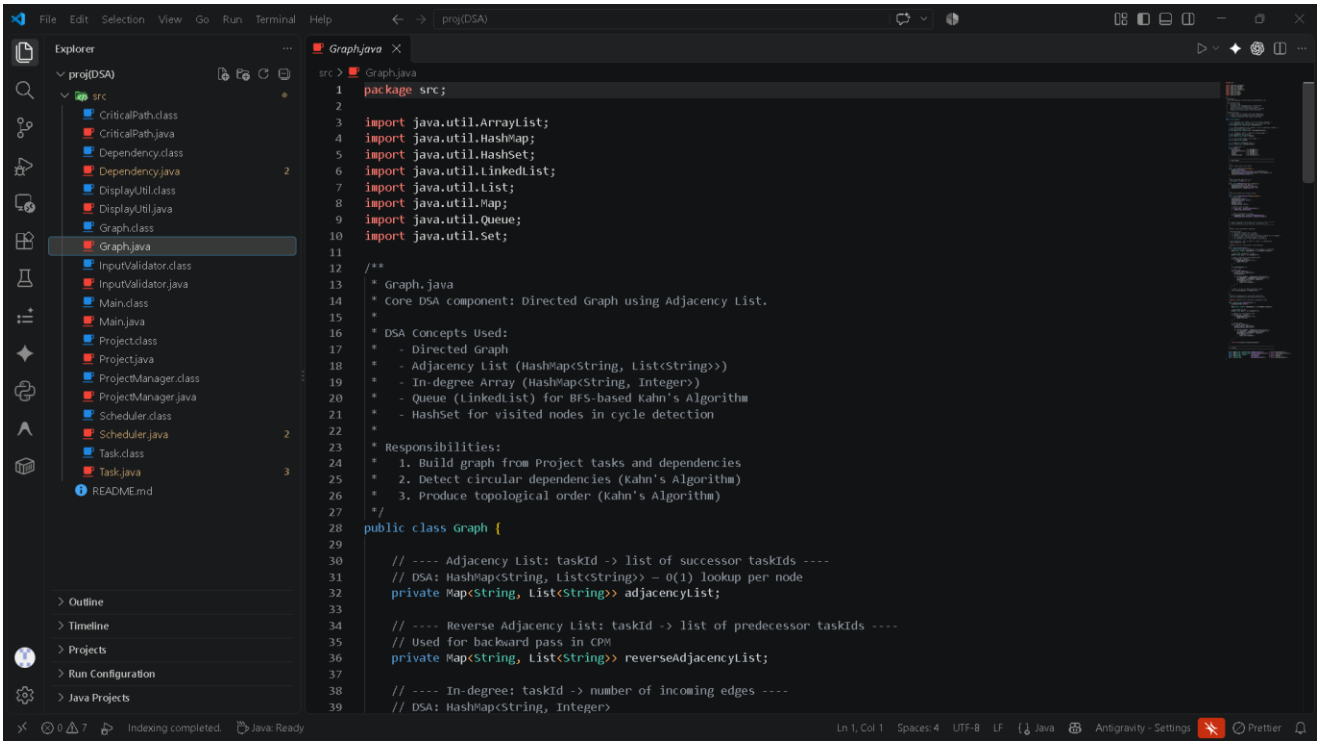
import java.util.List;
import java.util.Map;

/**
 * Scheduler.java
 * Implements the Critical Path Method (CPM).
 *
 * DSA Concepts:
 *   - Forward Pass: calculates Earliest Start Time (EST) & Earliest Finish Time (EFT)
 *   - Backward Pass: calculates Latest Finish Time (LFT) & Latest Start Time (LST)
 *   - Slack calculation: Slack = LST - EST
 *
 * Algorithm:
 *
 * FORWARD PASS (left to right in topological order):
 *   EST(task) = max(EFT of all predecessors)
 *   EFT(task) = EST(task) + duration
 *   Tasks with no predecessors: EST = 0
 *
 * BACKWARD PASS (right to left — reverse topological order):
 *   LFT(last task) = project duration
 *   LST(task)      = LFT(task) - duration
 *   LFT(pred)      = min(LST of all successors of pred)
 *
 * SLACK:
 *   Slack = LST - EST
 *   Critical Task: Slack == 0
 *
 * Time Complexity:  O(V + E)
 * Space Complexity: O(V)
 */
public class Scheduler {

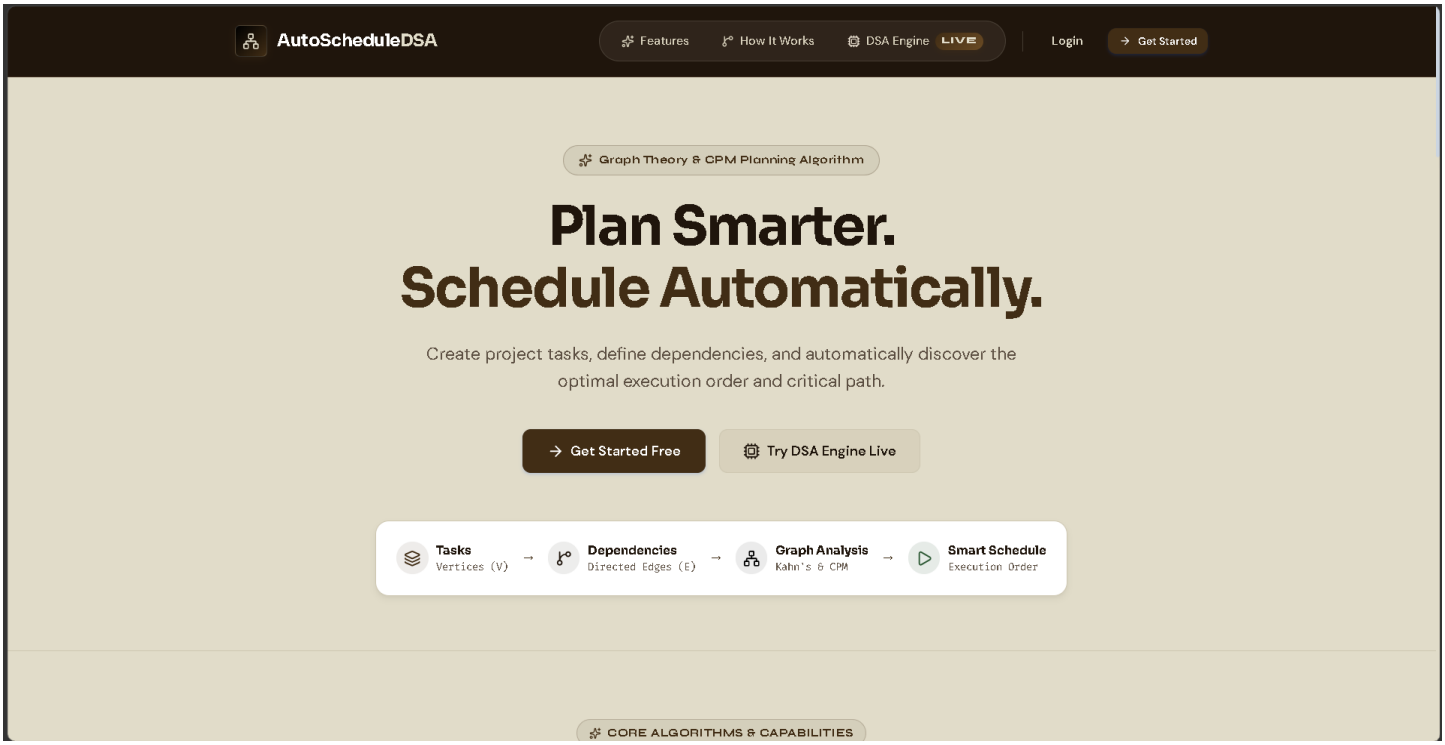
    private Project project;
    private Graph   graph;

    public Scheduler(Project project, Graph graph) {
```

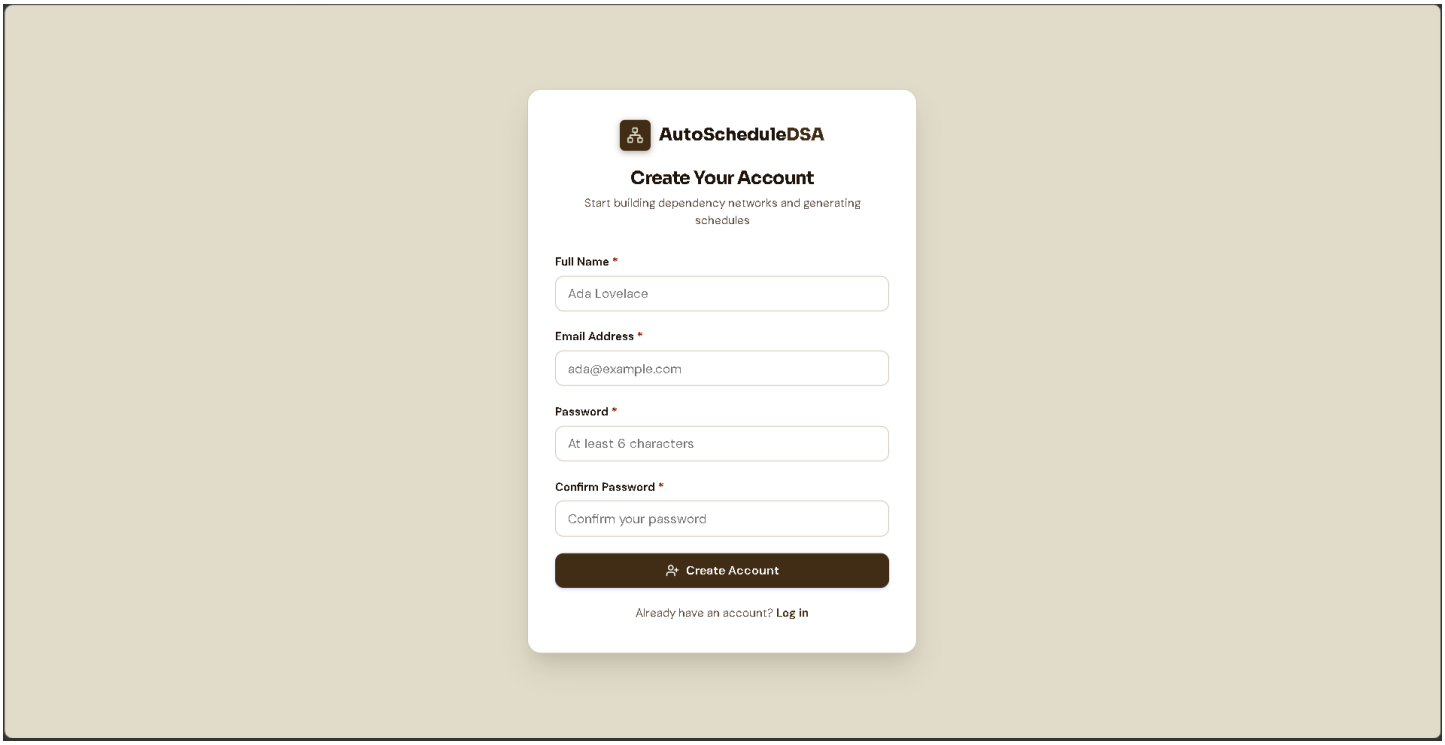




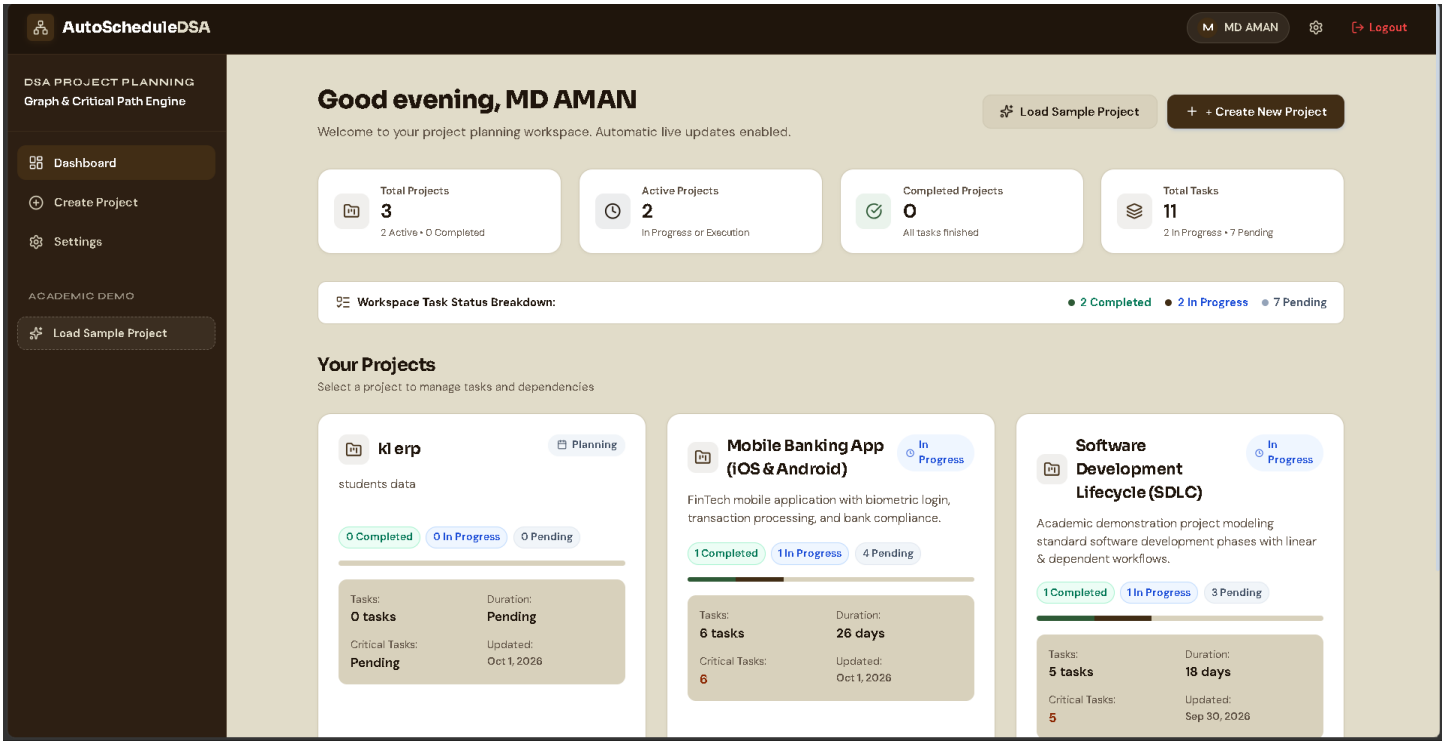
7. RESULTS & DISCUSSION



The homepage provides a clean and professional interface for creating projects, defining task dependencies, and generating schedules automatically. It clearly presents the workflow from **Tasks → Dependencies → Graph Analysis → Smart Schedule** for easy user understanding.



The registration page provides a simple account creation form with fields for name, email, password, and password confirmation. It allows new users to securely create an account before accessing the project scheduling features.



The dashboard provides an overview of all projects, tasks, and their current progress in a single interface. It allows users to create or load projects and quickly view project status, task counts, durations, and critical tasks.

8. CONCLUSION & FUTURE SCOPE

Conclusion:

The **Automated Project Task Planning and Scheduling** system successfully provides an organized approach to project scheduling by managing tasks, dependencies, and task durations. The system uses a directed graph, cycle detection, Topological Sorting, task timing, and Critical Path analysis to generate a valid execution order and identify critical tasks. It helps reduce manual scheduling effort and provides a clear view of the project completion process.

Future Scope:

The project can be further enhanced by adding:

- **Resource Management:** Assign people, equipment, and resources to tasks.
- **Real-Time Updates:** Automatically update schedules when task durations or dependencies change.
- **Progress Tracking:** Track completed, ongoing, and pending tasks.
- **Notifications:** Alert users about delays, deadlines, and critical tasks.
- **Advanced Optimization:** Support resource-constrained scheduling and more advanced scheduling techniques.
- **Larger Projects:** Improve scalability for projects with thousands of tasks and complex dependencies.

9. REFERENCES

- Microsoft Support – **Manage Your Project's Critical Path**
https://support.microsoft.com/en-us/project/manage-your-project-s-critical-path Microsoft Support

- Microsoft Support – **Show the Critical Path of Your Project**
https://support.microsoft.com/en-us/project/show-the-critical-path-of-your-project Microsoft Support

- Smartsheet – **How to Identify and Manage Dependencies in Project Management**
https://www.smartsheet.com/content/project-dependencies Smartsheet

- Atlassian Jira – **What Are Dependencies in Your Plan?**
https://support.atlassian.com/jira-software-cloud/docs/what-are-dependencies-in-advanced-roadmaps/ Atlassian Support

- Atlassian Jira – **View and Manage Dependencies in Your Plans**
https://support.atlassian.com/jira-software-cloud/docs/view-and-manage-dependencies-in-advanced-roadmaps/ Atlassian Support

- Atlassian Jira – **Add a Dependency in Your Plan**
https://support.atlassian.com/jira-software-cloud/docs/add-a-dependency-from-your-timeline-in-advanced-roadmaps Atlassian Support

- GeeksforGeeks – **Topological Sorting in Python**
https://www.geeksforgeeks.org/python/python-program-for-topological-sorting/ GeeksforGeeks

- ProjectLibre – **Open-Source Project Management Software**
https://projectlibre.com/

- ProjectLibre – **Linking Tasks and Dependencies**
https://projectlibre.com/academy/linking-tasks-and-dependencies/

- Lucidchart – **Critical Path Method (CPM)**
https://www.lucidchart.com/pages/critical-path-method